Distilled Command Parser on a Local Kubernetes Cloud

CS3081 Cloud Computing, Mini Project Report

Nishchay Sharma, Roll No. 230101030

GitHub: https://github.com/SharmaNishchay/cloud-project (tag `final`)

Status at submission

This report separates **measured results** (copied from my own command output) from **assumptions or plans**. The project is **only partly complete**. I ran out of time after infrastructure problems on my single laptop (Section 8).

Task	Status
1. Object storage and parallel teacher labelling	Cluster, storage and two parallel workers built and run. Labelling **not finished** (about 940 of 3,000 commands labelled). `clean.py` run on 780 saved records.
2. Student training	Code and config done. Only the 50-record smoke test ran. **No full v1/v2 model, no release, no three-model evaluation.**
3. Inference service in a container	Service and Dockerfile written, image built (1.26 GB). **50-request test not run.**
4. Kubernetes deployment and reliability	Manifests written. **No experiments run.**
5. Subnetting and NACL	**Complete**, 6/6 sample packets correct.
6. Reproducibility and CI/CD	Makefile, tests (9 passing locally), workflow and `AI_USE.md` written. **No failing-gate run link; secret scan not confirmed.**
7. Performance, cost, review	**Not done** (needs a deployed model).

1. Task 1: Object storage and parallel teacher labelling

Architecture. A k3d cluster (`cluster/k3d.yaml`, name `cloudmini`, one server node) runs SeaweedFS in namespace `data` with an S3 gateway (credentials generated at deploy time and kept only in Kubernetes Secrets). Buckets `datasets` and `models` exist, and `commands_pool.jsonl` is in `datasets`. The teacher is `phi3.5` served by Ollama on the laptop (temperature 0, seed 42), reached from the pods over the laptop's LAN address. The prompt is in `prompts/teacher_prompt.txt`: it defines the four-key JSON, the eight intents, the three slot rules and gives five worked examples.

Sharding (explanation). The worker (`worker/worker.py`) reads `SHARD_INDEX` and `SHARD_COUNT`. It numbers the lines of `commands_pool.jsonl` from 0 and keeps those where `line_number % SHARD_COUNT == SHARD_INDEX`. With `SHARD_COUNT=2`, worker 0 takes even lines and worker 1 takes odd lines, so the shards are disjoint and together cover every line exactly once. A Kubernetes Indexed Job in namespace `pipeline` runs two pods at the same time, and each pod gets its index from the Job completion index. Each worker writes its own object (`datasets/teacher/shard-0.jsonl`, `shard-1.jsonl`), storing the teacher reply exactly as returned. It saves a checkpoint every 20 records and resumes from it after a crash, which I needed several times. Note: Ollama handles requests one at a time by default, so two workers mainly gave concurrency and fault isolation, not a 2x speed-up.

Cleaning (`worker/clean.py`). It reads all shard objects, applies checks (a) invalid JSON, (b) keys not exactly `intent/date/time/place`, (c) intent not one of the eight, (d) a slot value that is neither null nor text found in the command, then (e) removes duplicate commands, keeping the smallest id. Records are written in id order and the SHA-256 of the output is stored. Running it twice on the same input gave the same checksum.

Measured counts. The cluster held about 940 labelled records when I had to stop. The counts below come from the 780 records I had saved to a local backup (`results/teacher_shard-0.jsonl`, `results/teacher_shard-1.jsonl`), so they describe a **partial** dataset.

Input	(a) invalid JSON	(b) wrong keys	(c) bad intent	(d) slot not in command	(e) duplicates	Kept
780	2	1	6	9	1	761

Check: 2 + 1 + 6 + 9 + 1 + 761 = 780. SHA-256 of this partial output: `831bf0e8e10ddbcbf948c821c8d69d37f6619f6a07a263dec557299fd661d878` (`results/clean_counts_partial.txt`). About 97.6% of the teacher replies were usable, so the prompt was good enough. Because labelling did not finish, `datasets/v1/train.sha256` for the full pool was **not** produced.

2. Task 2: Training the student (sequence-level distillation)

What exists. `training/train.py` fine-tunes `EleutherAI/pythia-70m` on CPU with PyTorch and Transformers. Each example is `Command: <text>` then `JSON: <teacher reply>`; the loss is computed only on the JSON part. `training/config.yaml` holds the seed (42), learning rate (3e-4), batch size (16), epochs (5), maximum sequence length (128) and the pinned library versions; `training/config_v2.yaml` is the second version (seed 7, 7 epochs). `make train` reads only the config and `make train SUBSET=50` trains on the first 50 records. Scripts to package the model (tarball plus SHA-256), upload it and score it exist (`scripts/publish_model.py`, `scripts/predict.py`).

Measured. I ran `make train SUBSET=50` to the end. Training loss per epoch (`training/loss_subset.csv`):

Epoch	1	2	3	4	5
Train loss	4.677	2.584	1.058	0.574	0.366

This only shows that the training code runs and learns on 50 examples. It says nothing about quality.

Not done (honestly). The full 1,500-record training (v1, v2), upload to `models/v1`, `models/v2`, the GitHub Release asset and `model.sha256` match, the scores of the teacher, untrained Pythia-70M and student on `validation.jsonl`, and `results/teacher_validation.json` were **not produced**. I therefore make no claim about validity or accuracy.

Sequence-level versus logit-based distillation. In sequence-level distillation (Kim and Rush, 2016) the teacher generates complete outputs, and the student is trained with ordinary cross-entropy on that generated text, as if it were the label. In logit-based distillation the student is trained to match the teacher's full probability distribution over the vocabulary at each token (for example by minimising KL divergence), which carries more information than the single chosen token. Logit-based distillation is not possible here because the teacher (`phi3.5` through Ollama) is used as a text-generation service and we only store its text. We have no access to its logits, and the two models use different tokenizers and vocabularies, so their distributions cannot be matched token by token. **One real limitation:** the student copies the teacher's mistakes. For example the teacher labelled "remind me of the meeting on tuesday" as `calendar_query`, although a reminder is more naturally `calendar_set`. The student can never be better than these labels, and a 70M model with 1,500 examples has little capacity on top of that.

3. Task 3: Inference service in a container

`service/app.py` (FastAPI, port 8000) provides `POST /parse` (4-key JSON with headers `X-Model-Version` and `X-Served-By`), `GET /healthz` (200 while alive) and `GET /readyz` (503 until the model is loaded). The model loads in a background thread, so the process stays alive while it downloads from object storage using `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT` and `MAX_NEW_TOKENS`; credentials come from a Secret. Every output is checked against `schema.json`; invalid output returns 500 with `{"error": "invalid_model_output"}`. `FAULT_INJECT=1` replaces the output with `not json` before the check. The Dockerfile uses pinned versions and contains no weights.

Measured: the image `cmd-parser:1.0` is **1.26 GB** (limit 2 GB). Four service unit tests pass (readiness 503, headers, fault injection returning 500 with the exact body, and broken output never returning 200). **Not done:** the 50-request test under 1 CPU and 1 GiB, and the peak-memory measurement.

4. Task 4: Kubernetes deployment, scaling and reliability

`k8s/30-app.yaml` defines namespace `app`, a Deployment (2 replicas, CPU and memory requests and limits, readiness probe on `/readyz`, liveness probe on `/healthz`, rolling update with `maxUnavailable: 0`), a Service, a Traefik Ingress for `/parse` and an HPA (min 1, max 2, 70% CPU). **None of the experiments were run** (rolling update and rollback, pod deletion, cold start, load balancing), so I report no results. **Limitation:** the cluster is a single node on one laptop, so it shows Kubernetes mechanics but not fault tolerance across machines, and node failure takes the whole system down. This actually happened to

me (Section 8).

5. Task 5: Subnetting and network access control (complete)

Allocation from `10.20.0.0/16`, largest first, each subnet placed right after the previous one (`network/subnets.md`):

Subnet	Hosts	Prefix	Network	Usable range	Broadcast	Usable hosts
app	1,000	/22	10.20.0.0	10.20.0.1 to 10.20.3.254	10.20.3.255	1,022
public	250	/24	10.20.4.0	10.20.4.1 to 10.20.4.254	10.20.4.255	254
data	100	/25	10.20.5.0	10.20.5.1 to 10.20.5.126	10.20.5.127	126
management	20	/27	10.20.5.128	10.20.5.129 to 10.20.5.158	10.20.5.159	30

Working: /23 gives only 510 hosts, so /22 (1,022) for app; /25 gives 126 (too few) so /24 (254) for public; /26 gives 62 so /25 (126) for data; /28 gives 14 so /27 (30) for management.

NACL (`network/nacl.json`, public subnet). Inbound: rule 50 denies `198.51.100.0/24` on all ports; rules 100 and 110 allow TCP 80 and 443 from anywhere; rule 120 allows TCP 22 only from the management subnet `10.20.5.128/27`; rule 130 allows replies (TCP 1024 to 65535) from the app subnet `10.20.0.0/22`. Outbound: rule 100 allows TCP 8000 to the app subnet; rule 110 allows replies to clients (TCP 1024 to 65535) anywhere. Everything else is denied by default. **Measured:** `tools/nacl_sim.py --check` gives 6 / 6 = 100%. Design note: because TCP 8000 lies inside the ephemeral range, outbound port 8000 to a non-app address is also allowed by rule 110.

Why replies need their own rules. A network ACL is stateless: it does not remember that a request was allowed, so every packet is judged alone against the rules for its direction. The reply to an allowed request is a separate packet to the client's ephemeral port, so it needs an explicit rule. **Why order matters:** rules are checked from the lowest number and the first match wins. The deny for `198.51.100.0/24` is rule 50, lower than the allows for ports 80 and 443 (100, 110). If it had a higher number, a packet from that range to port 443 would match the allow first and get through.

6. Task 6: Reproducibility, testing and CI/CD

The repository contains the `Makefile` (`make up`, `make down`, `make train`, `make test`, `make lint`), 9 unit tests (cleaning rules, sharding, service behaviour, NACL), `ruff` linting and a GitHub Actions workflow with the stages lint, unit tests, model evaluation (downloads the model from the GitHub Release and runs `score.py`), a quality gate (`score.py --min-validity 95`) and a gitleaks secret scan. Workflow input `model=base` runs the untrained Pythia-70M to show the gate failing. **Measured:** locally `make lint test` passes (9 tests). **Not done:** no GitHub model release exists, so the evaluation stage and the deliberate failing run were **not executed**; there is no link to a failed run, and I did not confirm a clean secret scan on GitHub. `AI_USE.md` is included and describes my AI use truthfully.

7. Task 7: Performance, cost and review

Not done. It needs a deployed model with a load test. I have no latency, throughput, error-rate, `kubectl top`, cost or SLO numbers and I do not estimate them. The cost method I would use, from the assignment: cost per 1,000 requests = (hourly cost of all running replicas) / (requests per hour) x 1,000, with each replica billed for its CPU limit and memory limit (`scripts/cost.py` computes this from a Locust result).

8. What went wrong, decisions and limitations

All problems were measured on my own machine (a MacBook Air running everything: Docker, the cluster, Ollama and training):

- Pods showed `ErrImagePull`: the locally built image was not inside the cluster. Fixed with `k3d image import`.
- Pods could not resolve `host.k3d.internal` and later `host.docker.internal`. Fixed by pointing workers

at the laptop's LAN IP.
- The Docker VM disk reached 100% (32 GB) and Kubernetes tainted the node with `disk-pressure`, so nothing could be scheduled. SeaweedFS' data directory had grown to 21 GB (as shown by `du`). I raised the Docker disk limit to 64 GB and the cluster recovered, and my labelled data survived.
- A `kubectl port-forward` dropped during a model upload. I added a retrying wrapper.
- The laptop reached a load average of 26.7 on 8 cores while Ollama, Docker and the cluster all ran together, so the API server timed out and the worker pods died. I ran out of time before the labelling and training could be finished.

Improvements I would make: start labelling much earlier; give Docker 64 GB and 6 GB RAM from the start; cap SeaweedFS volume size; run the teacher alone on the laptop and the cluster on lighter settings; test the full pipeline end to end on a small subset early. Assumptions are stated where they appear. All numbers in this report come from my own runs.

References

Kim and Rush, *Sequence-Level Knowledge Distillation*, EMNLP 2016. Bastianelli et al., *SLURP*, EMNLP 2020 (CC BY 4.0). EleutherAI, Pythia-70M model card (Apache 2.0).